

```
// SimpleStorage.sol
pragma solidity ^0.8.0;

contract SimpleStorage {
    uint256 storedData;

    function set(uint256 x) public {
        storedData = x;
    }

    function get() public view returns (uint256) {
        return storedData;
    }
}
```

Figure 3: A basic Solidity contract

be compiled into bytecode (the machine-readable code that runs on the Ethereum Virtual Machine). This is done using the Solc compiler.

This is how it works:

- Solidity (the code you wrote) is compiled into bytecode.
- The bytecode is the set of low-level instructions that can be executed by the Ethereum Virtual Machine (EVM).

You can compile your Solidity code using Remix IDE, a popular online tool for Solidity development, or use the Solc command-line compiler.

For example:

```
solc --bin SimpleStorage.sol
```

This command will output the bytecode that will be deployed to the blockchain.

Step 3: Deploy the bytecode to the Ethereum blockchain:

After compiling, the bytecode is ready to be deployed to the Ethereum blockchain. This is where the Ethereum Virtual Machine (EVM) comes into play.

- Deploying means uploading your smart contract's bytecode to the Ethereum network.
- This step is where the contract gets a permanent location on the blockchain and is ready to interact with users.

Here's a deployment example using Remix IDE or Truffle (a development framework for Ethereum):

- Connect to a wallet (e.g., MetaMask).
- Deploy the contract by submitting the bytecode to the Ethereum network.

The contract gets an address, like 0x1234567890abcdef..., which you'll use to interact with it later.

```
// Estimate Gas for set()
const gasEstimate = await contract.methods.set(42).estimateGas({ from: userAddress });
console.log(gasEstimate); // Will output the estimated gas
```

Figure 5: Gas estimate using a function

```
// Using web3.js to interact with the contract
const contractAddress = '0xYourContractAddress';
const abi = [ /* ABI of the contract */ ];

const web3 = new Web3(window.ethereum);
const contract = new web3.eth.Contract(abi, contractAddress);

// Store a number
await contract.methods.set(42).send({ from: userAddress });

// Retrieve the stored number
const value = await contract.methods.get().call();
console.log(value); // Should log 42
```

Figure 4: Example interaction

Step 4: Interact with the smart contract (calling functions): Now that your contract is deployed on the blockchain, users can interact with it. For example, using the `set()` function to store a value or the `get()` function to retrieve it.

Here's an example:

- When a user calls the `set()` function to store a value, that function executes and updates the state of the contract.
- Calling the `get()` function retrieves the current stored value.

Step 5: Gas fees – the cost of executing transactions:

Every time you interact with a smart contract, such as calling the `set()` or `get()` function, you'll need to pay gas fees. Gas is the computational cost required to run operations on the Ethereum network. The cost of gas is paid in ETH, and it compensates miners or validators for processing the transaction. More complex operations (like loops or storing large amounts of data) consume more gas.

For example:

- Calling the `set()` function to store a number consumes gas, and the more expensive the operation, the higher the gas fees.
- `get()` is a read-only function (doesn't change state), so it's free in terms of gas when using it locally. However, reading from the blockchain still costs a small fee in practice.

When calling a function, the gas used can be calculated by the EVM based on the computational complexity. Figure 5 shows how you can estimate gas using a function call in Remix IDE.

Step 6: Executing the transaction: Once the transaction is sent (and gas is provided), the EVM validates it and executes the smart contract. If there's enough gas, the transaction goes